



Ben-Gurion University of the Negev
The Faculty of Natural Sciences
The Department of Computer Science

Defending MCP Servers from Agents: A Static and Dynamic Risk-Scoring Framework for the Model Context Protocol

Research proposal submitted towards the requirements
for the Master of Sciences degree

Tomer Ovadya

Under the supervision of **Prof. Asaf Shabtai**

July 2026



Ben-Gurion University of the Negev
The Faculty of Natural Sciences
The Department of Computer Science

**Defending MCP Servers from Agents:
A Static and Dynamic Risk-Scoring
Framework
for the Model Context Protocol**

Research proposal submitted towards the requirements
for the Master of Sciences degree

Tomer Ovadya

Under the supervision of **Prof. Asaf Shabtai**

Signature of student: _____ Date: _____

Signature of supervisor: _____ Date: _____

Signature of chairperson of the
committee for graduate studies: _____ Date: _____

July 2026

Defending MCP Servers from Agents: A Static and Dynamic Risk-Scoring Framework for the Model Context Protocol

Tomer Ovadya

M.Sc. Research Proposal

Ben-Gurion University of the Negev

2026

Abstract

The Model Context Protocol (MCP) has become the standard interface through which autonomous large-language-model agents invoke external tools — filesystems, databases, code hosts, and messaging platforms. Almost all existing MCP-security research protects the *agent* from a malicious server. This research proposal inverts that direction and takes the standpoint of the **server as the protected asset and the agent as the threat source**: a server that fronts sensitive resources has no native way to decide, when a call arrives, whether it should be allowed, throttled, or denied, and an indirectly-injected agent is indistinguishable from a legitimate one.

We propose **McpRisk**, a defense-oriented risk-scoring framework that assigns every MCP tool interaction a graduated, interpretable four-band risk score. The score is a product of three interpretable factors — the sensitivity of the asset, the blast radius of the operation, and the impact (reversibility) of the tool, the last grounded in a 13-operation atomic-operation severity taxonomy — computed statically at design time and escalated at request

time by a parameter layer that reads the actual arguments of a call. The pipeline is driven by a single deterministic local model and audited by an independent review subsystem.

A proof-of-concept implementation and preliminary evaluation against four kinds of ground truth — realistic MCP deployments, a ten-rater human-and-framework oracle panel, a corpus of captured agent calls, and external attack benchmarks — already show that the approach is feasible: McpRisk agrees with reviewed judgments on tool impact 97% of the time and attains a quadratic-weighted κ of +0.66 against the human panel, and, unlike capability-only scorers, retains discriminative power on live external attacks. The preliminary results also expose the challenges the remaining research must solve, chiefly the fragility of the composite risk band. This proposal sets out the scoring model, these preliminary results, and a detailed research plan for developing a contextual request-time scorer and a deployable server-side risk gate.

Acknowledgements

First and foremost, I would like to thank my supervisor, Prof. Asaf Shabtai, for his guidance, his sharp questions, and his insistence on measuring what I claimed rather than asserting it. This thesis is far more honest, and far more useful, because of that discipline.

I am grateful to the members of the research group and the Department of Computer Science at Ben-Gurion University of the Negev for the discussions and feedback that shaped this work, and to the industrial collaborators who framed the practical constraints of cost, speed, and deployability that kept the project grounded in a real problem.

Finally, I thank my family and friends for their patience and support throughout this work.

Contents

List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Approach and Research Objectives	2
1.4 Proposal Structure	4
2 Background and Related Work	5
2.1 The MCP Attack Surface	5
2.2 MCP-Specific Defenses and Attacks	6
2.3 Safety of Tool-Using Agents and Prompt Injection	6
2.4 Guardrails and Runtime Gating	7
2.5 Risk Scoring: From CVSS to LLM Judges	7
2.6 Positioning of the Proposed Research	8
3 Proposed Approach: The McpRisk Framework	9
3.1 Threat Model	9

3.2	System Architecture	10
3.3	The Scoring Model	12
3.4	Atomic-Operation Taxonomy	13
3.5	Parameter (Request-Time) Scoring	14
3.6	Operationalizing Likelihood: Misuse Scoring of Call Logs . . .	15
3.7	Implementation and Reproducibility	17
4	Preliminary Results	18
4.1	Setup	18
4.2	Static Scoring vs. Human Judgment	19
4.3	Robustness, Request-Time, and Out-of-Distribution Detection	20
5	Research Plan and Expected Contributions	22
5.1	Summary of Progress	22
5.2	Remaining Challenges	23
5.3	Planned Work	23
5.4	Timeline	24
5.5	Expected Contributions	24
	Bibliography	25

List of Figures

3.1	The McpRisk pipeline. The scanner and static-scoring stages run at design time; the parameter and call-scoring stages run at request time. The review subsystem audits outputs independently of the scorer.	11
-----	---	----

List of Tables

3.1	The asset-sensitivity scale (s).	12
3.2	Risk-band calibration. Thresholds on the raw score are medium ≥ 8 and high ≥ 24 ; the rules below refine them.	13
3.3	The 13 atomic operations and their fixed severities.	14
4.1	Scanner vs. the human/framework panel, against the inter-rater ceiling. “Exact” and “ ≤ 1 ” are band-agreement rates. . .	19
4.2	Agreement with the human oracle (pooled). QW- κ is quadratic-weighted Cohen’s κ ; over-block is the fraction rated high/critical. . .	20
4.3	Attack-vs-valid detection on 150 external-benchmark calls. . .	21
5.1	Planned schedule for the remaining research.	24

Chapter 1

Introduction

1.1 Motivation

Large language models (LLMs) are increasingly deployed as *autonomous agents* that do not merely produce text but take actions in the world: they read and write files, query databases, push code, and send messages on a user’s behalf. The Model Context Protocol (MCP), introduced by Anthropic in late 2024 [1], has become the de facto standard for connecting such agents to external capabilities. An MCP *server* advertises a set of *tools* (for example `read_file`, `delete_file`, or `write_query`); an MCP *client*, driven by an LLM agent, discovers those tools and invokes them over a uniform JSON-RPC interface. This uniformity is precisely what makes MCP powerful — and what makes it dangerous.

The overwhelming majority of published MCP-security research studies the protocol from the *client’s* point of view: how a malicious or poisoned server can trick an agent into leaking data or executing harmful instructions [2, 3]. This thesis deliberately inverts that direction. We take the standpoint of the **server as the protected asset** and the **agent as the threat source**. A server that exposes a corporate filesystem, a production database, or a company Slack workspace has no native way to decide, at the moment a call arrives, whether `delete_file` on a private key or an unbounded `SELECT *`

FROM `api_tokens` should be allowed, throttled, or denied. An agent that has been indirectly prompt-injected [4, 5] looks, to the server, exactly like a legitimate one. The server therefore needs a way to reason about *how risky each incoming request is* before it executes.

1.2 Problem Statement

Traditional vulnerability-scoring systems do not answer this question. CVSS, DREAD, and the NIST risk methodologies were designed for static software and human-driven workflows; they score a *vulnerability*, not a live tool invocation, and they are argument-blind — they cannot distinguish `ls /tmp` from `rm -rf /`. Recent work confirms that when these scores are applied to AI-specific threats they collapse toward a single value and lose discriminative power [6, 7]. At the same time, the emerging MCP-specific defenses are largely *binary* detectors that label a request attack-or-benign [8, 9]; they do not give the server a graded, interpretable number it can threshold and act on.

This thesis asks: *Can we assign every MCP tool interaction a graduated, interpretable risk score — computed both statically from the tool’s declared properties and dynamically from the specific request — that a server can use to gate, throttle, or deny agent actions before they execute, under the threat model in which the server is the victim and the agent is the threat?*

1.3 Approach and Research Objectives

We propose to answer this question by designing, implementing, and evaluating **McpRisk**, a defense-oriented risk-scoring framework for MCP servers. McpRisk decomposes the risk of a tool interaction into a small set of interpretable factors — the *sensitivity* of the asset being touched, the *blast radius* of the operation, and the *impact* (reversibility) of the tool — and combines them into a four-band score (*low, medium, high, critical*). A *static* pipeline

scores each (tool, asset) pair at design time as an upper bound; a *parameter* layer then escalates that band at request time according to the magnitude of the actual arguments (for example, a **SELECT** with no **LIMIT** is treated as touching every row). Every score is produced by a local, deterministic LLM working against an explicit rubric, cross-checked by an independent LLM judge, and — crucially — validated against a panel of human and framework oracles rather than asserted.

The objectives of this research are:

1. A **server-centric threat model** for MCP (Section 3.1) that treats the agent as the adversary and formalizes the client $\rightarrow$ server attack direction, together with a mapping from MCP tools to a 13-operation *atomic-operation* severity taxonomy.
2. A **factor-based scoring model** (Section 3.3) that replaces the rigid CVSS base score with an interpretable product of sensitivity, blast radius, and impact, calibrated to a risk pyramid, together with a parameter-magnitude layer for request-time escalation.
3. A **proof-of-concept implementation** (Chapter 3) comprising a static scanner, a six-stage scoring pipeline, a parameter-rubric engine, and a call ranker, driven by a single reproducible local model.
4. A **multi-oracle evaluation** (Chapter 4) against realistic MCP deployments, a human-and-framework panel, a corpus of captured calls, and external attack benchmarks — reporting where the approach succeeds and where it does not.

Objectives 1–3 and an initial version of objective 4 have been realized in preliminary work (Chapters 3 and 4); the remaining evaluation and the improvements it motivates form the research plan of Chapter 5.

1.4 Proposal Structure

Chapter 2 reviews the work surrounding the problem: the MCP attack surface, MCP-specific defenses and attacks, the safety of tool-using agents, guardrails and runtime gating, and risk-scoring methodologies. Chapter 3 presents the proposed McpRisk threat model, the scoring model and its formulas, the atomic-operation taxonomy, and the pipeline. Chapter 4 reports preliminary results from a proof-of-concept implementation. Chapter 5 sets out the remaining challenges, a detailed research plan with a timeline, and the expected contributions.

Chapter 2

Background and Related Work

This chapter reviews the work most relevant to a defense-oriented risk scorer for the Model Context Protocol (MCP). We cover five areas: the MCP attack surface, MCP-specific defenses and attacks, the safety of tool-using agents, guardrails and runtime gating, and risk-scoring methodologies from CVSS to LLM-based judges. We then position the proposed research.

2.1 The MCP Attack Surface

MCP standardizes how LLM agents discover and invoke external tools, resources, and prompts exposed by a server [1]. The uniform interface that makes MCP powerful also creates a broad attack surface: any agent that speaks the protocol can request any exposed operation, and the server has little native machinery to decide whether a request should be allowed — a gap the protocol’s own security guidance acknowledges [10]. Hou et al. [11] give the first systematic map of this surface, defining the server lifecycle and a threat taxonomy across four attacker types, while Rostamzadeh et al. [12] organize attacks by the architectural component responsible for enforcement and find protection uneven and predominantly tool-centric — directly motivating a server-side gate. Empirical studies confirm the risk is real: Radosevich and Halloran [13] coerce models through MCP tools into compromising

a host; Hasan et al. [14] find that of 1,899 servers, 7.2% carry general vulnerabilities and 5.5% exhibit tool poisoning; Zhao et al. [15] measure 12,230 tools across 1,360 servers and define parasitic toolchain attacks; and Li et al. [16] statically analyze 2,562 servers and argue for explicit privilege management. Yang et al. [9] standardize this into a benchmark of seventeen attack types and report that existing protections block fewer than 30% of attacks.

2.2 MCP-Specific Defenses and Attacks

On the defensive side, Xing et al. [8] propose *MCP-Guard*, a proxy-based three-stage pipeline (static scan, neural detector, and LLM arbitrator) whose layered architecture is the closest in spirit to ours, and Jing et al. [3] track contextual integrity of information flows. Because a tool’s natural-language description drives agent tool selection, that description is itself an attack vector: Shi et al. [2] show a single poisoned tool document can hijack tool selection, and tool-squatting and rug-pull attacks motivate OAuth-signed, versioned tool definitions [17]. The work closest to ours places enforcement at the server boundary: Bühler et al. [18] present the first access-control framework for MCP servers, Sharma et al. [19] propose precise task-scoped authorization in which the server verifies a call is a legitimate consequence of the user’s task, and Huang et al. [20] measure “caller identity confusion” — servers generally cannot tell which agent is invoking a request. All three issue an allow/deny decision; our proposal instead computes a graduated risk *score*.

2.3 Safety of Tool-Using Agents and Prompt Injection

Because the threat originates from an agent, the agent-safety literature characterizes what a server-side scorer must anticipate. Yuan et al. [21] show that even strong models judge the safety of agent actions correctly only 74%

of the time; Andriushchenko et al. [22] find leading models compliant with malicious agent tasks; and Ruan et al. [23] use LM-emulated sandboxing to measure risky behavior at scale. Most pointedly for us, Cartagena and Teixeira [24] demonstrate that textual safety does *not* transfer to tool-call safety — models often refuse in chat yet still execute the harmful call — a direct argument for scoring the call itself. Indirect prompt injection is the mechanism that turns a benign agent into a threat: Zhan et al. [4] and Debenedetti et al. [5] show that tool-returned data can hijack agent behavior in realistic workflows. Rather than detecting injection, our proposal bounds the *damage* an injected agent can do by scoring each requested operation before it runs.

2.4 Guardrails and Runtime Gating

A growing line of work places an interpretable enforcement layer between a model and its effects, from LLM-based input/output classifiers [25] to per-call least-privilege systems: Progent expresses tool-call permissions as verifiable symbolic rules [26], and AgenTRIM reconstructs tool interfaces offline and blocks misaligned calls at runtime [27]. These systems decide *whether* to allow an action; our framework is complementary in computing *how risky* an action is on a graduated scale, a signal such gates could consume. The need is underscored by evidence that agents can behave as strategic “insider threats” when their goals are obstructed [28].

2.5 Risk Scoring: From CVSS to LLM Judges

Our framework is, at its core, a scoring system. Spring et al. [7] argue CVSS is inadequate as a risk proxy because its algorithm is neither formally nor empirically justified; Jacobs et al. [29] complement severity with data-driven exploitation likelihood; and Bahar and Wazan [6] show CVSS, DREAD, and OWASP Risk Rating produce minimal variation across very different LLM attacks. The community has responded with agentic-specific scores and check-

lists [30–32], and with LLMs applied directly to security scoring: prior work from our own group uses LLMs to detect and reason about misconfigurations (GenKubeSec [33], KubeGuard [34]), and others evaluate LLMs for vulnerability triage [35]. The mechanism we adopt — an LLM producing a numeric score against an explicit rubric — descends from the LLM-as-judge literature: G-Eval couples chain-of-thought with form-filling for calibrated scores [36], and Zheng et al. [37] both validate the approach and catalog its biases. Two very recent works retarget this to our exact setting — SkillVetBench scores agent skills with a CVSS-flavored rubric [38], and Fleming et al. [39] have an LLM judge each access request to drive risk-adaptive access control — and, like them, we validate the LLM’s scores against external oracles rather than trusting them blindly. Finally, since an MCP call is structurally an authenticated API request, the runtime half of our framework draws on dynamic risk assessment [40].

2.6 Positioning of the Proposed Research

Three gaps recur across this literature. First, the fast-growing set of MCP defenses is overwhelmingly *binary* (attack vs. benign) and tool-centric, leaving no graded, server-side notion of “how risky is this particular call.” Second, general vulnerability scores such as CVSS are demonstrably insensitive to AI-specific threats. Third, even the systems closest to ours — AgentBound [18] and PAuth [19] — issue an allow/deny decision rather than a graduated risk score. The proposed research addresses all three by placing an interpretable, factor-based risk scorer at the server boundary, computed statically from a tool’s declared properties and dynamically from the specific request.

Chapter 3

Proposed Approach: The McpRisk Framework

This chapter presents **McpRisk**, our defense-oriented risk-scoring framework. We first make the threat model precise (Section 3.1), then describe the end-to-end system architecture (Section 3.2), the factor-based scoring model and its formulas (Section 3.3), the atomic-operation severity taxonomy (Section 3.4), the parameter layer that turns the static score into a request-time score (Section 3.5), and finally the implementation and its reproducibility guarantees (Section 3.7).

3.1 Threat Model

McpRisk adopts a single, deliberately asymmetric threat model:

The MCP server is always the protected asset; the agent is always the threat source.

Every interaction we consider flows *client* $\rightarrow$ *server*: an autonomous or semi-autonomous LLM agent (or the user behind it) issues tool calls through the protocol in order to read, modify, destroy, or exfiltrate resources that

the server fronts. The agent may be malicious by intent, or benign but compromised — for instance, an agent that has ingested an indirect prompt injection [4] and now acts against the server’s interest. From the server’s vantage point these cases are indistinguishable, so McpRisk does not attempt to infer intent; it scores the *consequence* of the requested operation.

The inverse scenario — a malicious server attacking its client, the classical “tool poisoning” direction [2] — is explicitly out of scope. Tool poisoning enters our model only as a *cause*: a poisoned agent subsequently becomes a threat *to the server*, and it is that downstream request we score. Two further design commitments follow from this framing. First, McpRisk is **not anomaly detection**: it does not flag deviations from a learned baseline of “normal” traffic, because a first-of-its-kind malicious call has no baseline. Instead it defines, from concrete factors, what *makes* an interaction risky. Second, the score is **graduated, not binary**: rather than an allow/deny verdict, it produces a four-level band so that a server can gate the top band, throttle or escalate the middle bands, and log the rest.

3.2 System Architecture

McpRisk is organized as a pipeline of loosely coupled stages, summarized in Figure 3.1. It operates in two phases that mirror the two scoring modes of the introduction.

Design time (static). A *scanner* runs before the server goes live. It never connects to a running server and reads no pre-existing risk table; instead it assembles a registry from two static sources — the tools the server advertises in its own `tools/list` response, and the assets of the store it fronts (a filesystem tree, a SQLite schema, or a set of declared Slack channels, calendars, or repositories). A six-stage *static-scoring* pipeline then assigns each (tool, asset) pair a risk band, producing a scan artifact.

Request time (dynamic). A *parameter-scoring* module derives, per tool, a rubric for the magnitude-bearing arguments of a call, and a *call-scoring*

module resolves each captured call to a scanned (tool, asset) cell and escalates its band according to the actual argument magnitudes. Calls that cannot be resolved are handled honestly: an unknown tool is marked *invalid* (a misconfiguration) and a known tool aimed at an unscanned target is marked *unresolved* (the scorer abstains) rather than guessing.

A separate *review* subsystem — comprising deterministic verification, an independent LLM judge, an advisor, and a panel of persona critics — audits the pipeline’s outputs but is never imported by the scorer itself, keeping evaluation independent of scoring.

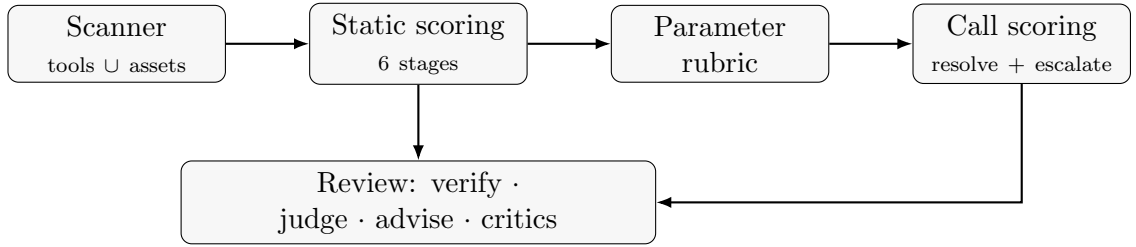


Figure 3.1: The McpRisk pipeline. The scanner and static-scoring stages run at design time; the parameter and call-scoring stages run at request time. The review subsystem audits outputs independently of the scorer.

The static-scoring pipeline itself proceeds through six stages, each of which prefers a local LLM working against an explicit rubric and degrades to a deterministic fallback when the model is unavailable (except in *strict* mode, where an unreachable model raises an error rather than fabricating a value): (0) *domain inference* of the server kind and profile; (1) *tool impact* on a 1–3 scale; (2) *asset sensitivity* on a 1–5 scale; (3) *blast radius* per (tool, asset) pair; (4) *baselines* of expected behavior and *risk-band* assignment; and (5) an independent *judge* that re-derives every primitive from the same domain profile and whose value overrides the proposer’s on disagreement. Over the seven evaluation servers this judge recorded 52 overrides, and the scan records a cross-check summary and provenance flag (**llm-scan** vs. **offline-baseline**) for every cell.

3.3 The Scoring Model

At the conceptual level, McpRisk views the risk of an interaction as the product of how much damage it can do and how likely that damage is:

$$\text{Risk} = \text{Impact} \times \text{Likelihood} \times \text{Irreversibility}. \quad (3.1)$$

In the implemented *static* pipeline, likelihood is pinned to its upper bound (Likelihood = 1.0) because a design-time score must assume the worst case, and irreversibility is folded into the tool-impact term. This yields the concrete static score for a (tool, asset) cell:

$$\text{score} = \underbrace{s}_{\text{sensitivity}} \times \underbrace{b}_{\text{blast radius}} \times \underbrace{i}_{\text{tool impact}}, \quad s \in [1, 5], b \in [0, 4], i \in [1, 3], \quad (3.2)$$

so the score ranges over $[0, 60]$. The three primitives are defined as follows.

Sensitivity s (1–5) measures the confidentiality and value of the asset being touched, from public data to regulated PII (Table 3.1). **Blast radius** b measures how far a single action propagates, from a single record (local) to the entire system (systemic). **Tool impact** i (1–3) measures reversibility: 1 for read-only tools that cannot change state, 2 for tools that make a recoverable change (write, edit, insert, move, post), and 3 for irreversible or access-changing tools (delete, drop, overwrite, execute, revoke, grant). This last term is where the framework encodes the intuition that `file_write` is inherently riskier than `file_read` and that `delete_file` is riskier still.

Table 3.1: The asset-sensitivity scale (s).

s	Asset class	Examples
1	Public / non-sensitive	Public config, non-sensitive logs
2	Low-sensitivity infrastructure	Generic file store, API endpoints
3	Internal operational data	Internal DB, system config, QA plans
4	Restricted business data	Audit logs, source code, email
5	Regulated / PII	Financial records, PII, private keys

The numeric score is discretized into four **risk bands** — *low*, *medium*, *high*,

critical — using the thresholds and calibration rules of Table 3.2. The calibration is deliberately shaped like a *risk pyramid*: only a cell that is irreversible ($i = 3$), aimed at a crown-jewel asset ($s = 5$), with at least departmental reach ($b \geq 3$) is labeled *critical*, so that in practice only 1–2% of cells reach the top band. Two special rules encode confidentiality intuitions that the raw product would miss: a broad read of highly sensitive data ($i = 1, s \geq 4, b \geq 3$) is treated as mass exfiltration and raised to *high*, and any read of a crown-jewel asset ($i = 1, s = 5$) is never allowed to fall below *medium*. Read-only operations otherwise remain *low*, so that ordinary work is not gated.

Table 3.2: Risk-band calibration. Thresholds on the raw score are medium ≥ 8 and high ≥ 24 ; the rules below refine them.

Band	Condition
critical	$i = 3 \wedge s = 5 \wedge b \geq 3$
high	$(i = 3 \wedge s \geq 4) \vee \text{score} \geq 24 \vee (i = 1 \wedge s \geq 4 \wedge b \geq 3)$
medium	$\text{score} \geq 8 \vee (i = 1 \wedge s = 5)$
low	otherwise

3.4 Atomic-Operation Taxonomy

To assign the tool-impact term consistently across heterogeneous servers, we map every MCP tool to one or more of thirteen *atomic operations*, each carrying a fixed severity from 1 (lightweight discovery) to 5 (total compromise), shown in Table 3.3. A tool’s effective impact derives from the highest-severity operation it can perform: a SQLite `write_query` tool that accepts free-form SQL, for example, is tagged with the full worst-case set (EXECUTE, DELETE, OVERWRITE, SCHEMA_MODIFY, WRITE) because the argument alone determines which it performs.

The mapping is produced by a *rules-based* classifier that runs two independent paths — one over the server’s README prose and one over the structural signals of its tool-list schema (name prefixes, parameter shapes) — and surfaces any disagreement between them for review. Keeping this stage rule-

based rather than model-based makes the tool-to-severity mapping auditable and fully reproducible.

Table 3.3: The 13 atomic operations and their fixed severities.

Operation	Severity	Rationale
EXECUTE	5	Arbitrary code/command execution; total compromise
DELETE	5	Irreversible data destruction
OVERWRITE	4	Corrupts existing data in full
SCHEMA_MODIFY	4	Structural damage; cascading failures
BROADCAST	4	Sends data externally; exfiltration + impersonation
WRITE	3	Injects new data; malicious persistence
MODIFY	3	Tampers with existing data; integrity violation
MOVE	3	Relocates resources; can hide or expose data
CREATE	3	New persistent resource; staging for writes
READ	2	Accesses content; confidentiality breach
SEARCH	2	Targeted discovery of sensitive data
METADATA	1	Attributes without content; structural recon
LIST	1	Enumerates resources; existence discovery

3.5 Parameter (Request-Time) Scoring

The static cell score is an upper bound over all possible invocations of a tool. To turn it into a request-time score, McpRisk adds a *parameter* layer that reads the actual arguments of a call. For each tool, the LLM derives a rubric that identifies the magnitude-bearing parameters, and for each such parameter a **base_rank** (its intrinsic sensitivity), an extraction rule (whether the magnitude is a number, a list length, a parsed **LIMIT**, or a boolean), and a set of ascending numeric **cutoffs**. A concrete value is banded by the highest cutoff it meets. A crucial rule is that the *absence* of a **LIMIT** clause is treated as unbounded (∞) and therefore worst-case, so that a **SELECT** or **DELETE** touching every row is scored as maximally broad.

Two bands are then combined arithmetically, where low = 1, medium = 2, high = 3, critical = 4. The *parameter risk* is the average of the parame-

ter's base rank and its value band, rounded half-up:

$$\text{param_risk} = \left\lfloor \frac{\text{base_rank} + \text{value_band}}{2} + 0.5 \right\rfloor. \quad (3.3)$$

The final band of the call escalates the inherent (tool, asset) band by the parameter risk, taking the more severe of the two so that a risky argument can only *raise* the band, never lower it:

$$\text{final_band} = \max(\text{cell_band}, \text{param_risk}). \quad (3.4)$$

For example, a `create_event` call whose `attendees` parameter has base rank *medium* and 50 recipients (value band *critical*) yields $\text{param_risk} = \lfloor (2 + 4)/2 + 0.5 \rfloor = 3$; combined with a *high* inherent cell this escalates the call to *critical*. When several parameters apply, the most severe governs.

3.6 Operationalizing Likelihood: Misuse Scoring of Call Logs

The parameter layer captures the *magnitude* of a request, but the conceptual model of Equation (3.1) also has a *likelihood* term. To score a captured call log end to end, McpRisk instantiates that term with a concrete formula, calibrated against the canonical lookup tables used to build the project's hand-labeled heatmaps:

$$\text{AssetImpact} = \min(\text{Sensitivity} \times \text{Blast}, 25), \quad (3.5)$$

$$\text{Likelihood} = \text{BaseDev} \times \text{Anomaly} \times \text{Exposure}, \quad (3.6)$$

$$\text{Misuse} = \max(\text{AssetImpact} \times \text{Likelihood} \times \text{ToolImpact}, 1.0), \quad (3.7)$$

where $\text{ToolImpact} \in \{1, 2, 3\}$ is the reversibility multiplier of Section 3.3. The three likelihood sub-factors follow the definitions of the conceptual model: *baseline deviation* (how unusual the call is relative to intended use), *access anomaly* (time/volume outlieriness), and *exposure* (how reachable the desti-

nation makes the asset). Crucially, the first two require a history of prior calls that a server gating a request in isolation does not have; consistent with the framework’s open question about how much runtime log context is available, they are set to conservative constants ($\text{BaseDev} = 0.9$, a high prior that any logged call may deviate; $\text{Anomaly} = 0.3$, a low value reflecting the absence of observable access patterns), while *exposure* is read from a lookup table keyed on the destination directory (from 0.3 for public areas to 0.9 for security-evidence stores). The floor of 1.0 ensures every logged call registers at least baseline activity. Because the context multiplier of the original design was removed, the band thresholds are rescaled accordingly to $\text{critical} \geq 27$, $\text{high} \in [13, 26]$, $\text{medium} \in [7, 12]$, and $\text{low} < 7$.

Five judgment rules close gaps the lookup tables leave open, and encode the threat intuitions a naive product would miss. (i) For cross-path tools (move/copy/rename), sensitivity is taken from the *source* and exposure from the *destination*; if a sensitive source ($\text{Sensitivity} \geq 4$) is moved to a low-exposure public area, exposure is forced to 1.0 — the asset is now reachable to anyone, and renaming its extension does not lower its sensitivity. (ii) A path-traversal attempt (e.g. `../../../../etc/passwd`) is scored as a system asset ($\text{Sensitivity} = 5$, $\text{Exposure} = 0.9$) even if it was blocked, because the probe itself is the signal. (iii) For multi-path calls, the worst case across paths governs. (iv) Errors and denied calls still score, since the attempt is a signal. (v) Asset-name overrides raise sensitivity where the extension hides it — an `audit_log.txt` or a file matching `*credential*` is scored at sensitivity 5 regardless of its `.txt` suffix. These rules are exactly the cases where the static cell score under-weights a real threat, and folding them into the request-time score is how the dynamic mode adds value over the static upper bound. A richer contextual scorer that learns baseline deviation and access anomaly from real runtime logs remains future work, as we discuss in Chapter 5.

3.7 Implementation and Reproducibility

McpRisk is implemented in Python 3.12 as the package `mcp_security`, with the stages of Figure 3.1 realized as the sub-packages `scanner`, `static_scoring`, `param_scoring`, `call_scoring`, `atomic_ops`, and `review`. All model-backed stages call a single local model — Qwen2.5:32B served through Ollama — with no cloud fallback by design, so that no server metadata leaves the machine. Decoding is made deterministic and reproducible by fixing greedy decoding parameters (temperature 0, seed 0, top- p 1.0). Because the same model both proposes and (through the judge stage) cross-checks scores, and because it also underlies part of the evaluation, we treat model-induced circularity as a first-class limitation and address it in Chapter 4 by validating against *external* human and framework oracles.

Chapter 4

Preliminary Results

To establish that the proposed approach is feasible and worth pursuing, we have implemented a proof-of-concept of McpRisk and run an initial evaluation. This chapter reports those *preliminary* results; a fuller evaluation is part of the planned work in Chapter 5. Because risk banding is intrinsically subjective, we do not evaluate against a single “correct” answer but triangulate across four kinds of ground truth, and we report both the results that favor the approach and those that do not.

4.1 Setup

The evaluation uses four datasets. First, **seven realistic MCP deployments** — corporate, legal, medical, and media filesystems, and two SQLite databases — each seeded with crown-jewel assets (private keys, payslips, patient records, API tokens); their judge-reviewed static tables (61 tools, over 40 assets, 5 *critical* cells) form the design-time ground truth. Second, an **independent ten-rater oracle panel** per cell, combining a hand-authored human heatmap with nine framework baselines (DREAD, CVSS v3, two NIST methods, OWASP Risk Rating, OWASP AIVSS, MAESTRO, and two ChatGPT raters); the scanner’s own model is excluded to avoid circularity. Third, a **captured-call corpus** of 969 calls from seeded simulations across

Calendar, GitHub, Slack, a fintech filesystem, and a DevOps database, mixing benign operators with attackers. Fourth, 150 labeled calls from three **external attack benchmarks** (MCPSecBench, MCP-SafetyBench, MSB) that share our client $\rightarrow$ server threat model.

All scores were produced by a single local model (Qwen2.5:32B via Ollama, temperature 0, seed 0). Since the output is an ordinal four-band label, we report *exact* and *within-one-band* agreement, quadratic-weighted Cohen’s κ , and — for the detection experiments — the area under the ROC curve (AUC). Panel consensus is the upper-median across raters, robust to the maximalism of frameworks such as DREAD that rate almost everything dangerous.

4.2 Static Scoring vs. Human Judgment

The two *primitive* judgments are highly consistent with the reviewed tables: tool-impact agreement is 59/61 (97%) and asset-sensitivity agreement 5/7 (71%). Agreement on the *composite* band is lower (34% exact), the first sign that the composite score — not the primitives — is the weak point. Against the independent panel, the crucial context is the *legitimate-disagreement ceiling*: the human and framework raters agree with *each other* only 50% (filesystem) and 42% (SQLite) of the time exactly (Table 4.1). Risk banding is inherently subjective, and no scorer can exceed this ceiling. McpRisk reaches 35% exact and 69% within-one agreement with the consensus — inside the band of human disagreement, though at its lower end — and tends to score *below* the conservative consensus, i.e. it under-alarms rather than over-alarms.

Table 4.1: Scanner vs. the human/framework panel, against the inter-rater ceiling. “Exact” and “ ≤ 1 ” are band-agreement rates.

Comparison	Exact	≤ 1 band
Inter-rater ceiling (filesystem)	50%	78%
Inter-rater ceiling (SQLite)	42%	75%
Scanner vs. consensus (overall)	35%	69%

Scoring the same cells with every baseline and comparing against the human oracle by quadratic-weighted κ (Table 4.2), McpRisk attains $\kappa = +0.66$ with 97% within-one agreement — second only to OWASP AIVSS and well ahead of CVSS v3. The maximalist frameworks reach apparent safety trivially by rating almost everything dangerous (DREAD $\kappa = +0.10$ at a 100% over-block rate); McpRisk’s 56% over-block matches AIVSS while preserving discrimination, the intended behavior of the risk pyramid.

Table 4.2: Agreement with the human oracle (pooled). QW- κ is quadratic-weighted Cohen’s κ ; over-block is the fraction rated high/critical.

Scorer	QW- κ	Within-1	Over-block
OWASP AIVSS	+0.69	—	56%
McpRisk (ours)	+0.66	97%	56%
CVSS v3	+0.36	—	—
ChatGPT (plain)	+0.23	—	98%
DREAD	+0.10	—	100%

4.3 Robustness, Request-Time, and Out-of-Distribution Detection

The approach’s most significant weakness is the *stability* of the composite score. Perturbing each of the 1,700 scored cells’ inputs by ± 1 , one at a time, changes the band for 85% of cells, with the fragility clustered at the *medium* threshold. The multiplicative formula with fixed integer cut-points is thus unstable precisely where a gate’s behavior hinges — the primary target of the planned work.

On the captured-call corpus, McpRisk produces a scorable band for 87% of a labeled subset, abstaining honestly on the rest, and reaches an AUC of 0.64 against the sessions’ authored-intent labels. These labels are a *weak* ground truth, so the numbers are diagnostic; the informative failure mode is that 27 malicious *read-only* exfiltration calls score only low/medium, since the inherent cell under-weights pure exfiltration — exactly the gap the planned

contextual scorer must close. The out-of-distribution result is the most encouraging (Table 4.3): on the external benchmarks, content-aware scoring separates attacks (LLM-judge AUC 0.68) while the argument-blind capability frameworks CVSS and AIVSS score 0.52 — essentially no skill, because they cannot see the arguments that make a call malicious. This is direct evidence for the proposal’s premise that risk must be assessed from the request content, not the tool’s capability alone.

Table 4.3: Attack-vs-valid detection on 150 external-benchmark calls.

Scorer	AUC	Recall @ 10% FPR
LLM judge (content-aware)	0.68	42%
Keyword (content-aware)	0.67	38%
CVSS (capability-only)	0.52	0%
AIVSS (capability-only)	0.52	0%
Random baseline	0.48	11%

Finally, an independent review subsystem audits the pipeline: a deterministic verifier passes all 8/8 correctness checks and a methodology judge confirms all 7/7 integrity claims (no data leakage, LLM-only with no fabrication, honest handling of unresolved calls), while a results judge returns a sober verdict of “adequate, with concern” — the pipeline is correct and honest, but its agreement with the human consensus is only modestly above chance. These preliminary findings, and the weaknesses they expose, motivate the research plan of the next chapter.

Chapter 5

Research Plan and Expected Contributions

This chapter summarizes the progress made so far, identifies the challenges that remain, and lays out a concrete plan — with work packages, a timeline, and expected contributions — for completing the research.

5.1 Summary of Progress

The preliminary work has established that interpretable, server-side risk scoring for MCP is feasible. We have (i) formalized the server-as-victim threat model (Section 3.1); (ii) designed the factor-based scoring model and its atomic-operation taxonomy (Sections 3.3 and 3.4); (iii) implemented the full static and request-time pipeline driven by a single deterministic local model; and (iv) run an initial multi-oracle evaluation (Chapter 4). The results are encouraging where the problem is well-defined — 97% agreement with reviewed judgments on tool impact and a quadratic-weighted κ of +0.66 against a human panel, second among eight frameworks — and, importantly, show that content-aware scoring separates live attacks where capability-only scores such as CVSS do not. This is sufficient evidence that the direction is worth completing.

5.2 Remaining Challenges

The preliminary evaluation also exposed the problems the remaining research must solve. The most significant is the *fragility of the composite band*: 85% of scored cells change band under a single ± 1 perturbation of their inputs, concentrated at the *medium* threshold, which undermines a gate that must act on the band. Second, the request-time evaluation rests on *weak, intent-based labels* (AUC 0.64), and the static model systematically under-weights read-only exfiltration. Third, calls are scored *individually*, so cumulative risk across a sequence of low-risk actions is not yet modeled. Fourth, reliance on a *single local model* for both scoring and part of the evaluation introduces a circularity we mitigated but did not eliminate. These four challenges define the work packages below.

5.3 Planned Work

WP1 — Stabilize the banding formula. Replace the hard integer cut-points with calibrated, monotone soft thresholds, or learn the band boundaries against the human consensus, and re-run the sensitivity analysis to target a substantial reduction in the 85% fragility.

WP2 — A contextual request-time scorer. Build the dynamic scorer that reads the actual arguments and recent call history to close the read-only-exfiltration gap, operationalizing the likelihood sub-factors of Section 3.6 from real runtime logs rather than fixed constants. The external-benchmark result suggests this is where the discriminative power lies.

WP3 — Multi-step chain modeling. Extend the scorer to accumulate risk across a session, so that reconnaissance followed by a single exfiltration is scored as the campaign it is rather than as isolated low-risk calls.

WP4 — Strengthened, multi-model evaluation. Replace the weak dynamic labels with an audited ground truth, broaden the oracle panel, and repeat the study across multiple model families to remove the single-model

circularity and test generalization.

WP5 — Deployment study. Integrate the scorer as a gate in front of a live MCP server and measure end-to-end latency and overhead, addressing the practical constraints of cost and speed that a real deployment imposes.

5.4 Timeline

We plan the remaining work over four phases, as summarized in Table 5.1. WP1 and WP2 are prioritized because they address the two weaknesses that most limit a deployable gate; WP3–WP5 build on their outputs.

Table 5.1: Planned schedule for the remaining research.

Phase	Focus
Months 1–3	WP1: formula stabilization and re-analysis
Months 3–7	WP2: contextual request-time scorer
Months 6–9	WP3: multi-step chain modeling
Months 8–11	WP4: strengthened, multi-model evaluation
Months 10–12	WP5: deployment study and thesis writing

5.5 Expected Contributions

On completion, the research is expected to contribute: (i) a server-centric threat model and a factor-based, interpretable risk-scoring model for MCP tool interactions; (ii) a stabilized banding formula with quantified robustness; (iii) a contextual request-time scorer that closes the exfiltration gap and models multi-step campaigns; (iv) a strengthened, multi-model evaluation against human and framework oracles; and (v) a deployable proof-of-concept gate with measured overhead. Together these would provide the first graduated, interpretable, server-side risk-scoring framework for the Model Context Protocol, giving servers a computable signal to gate, throttle, or deny risky agent requests before they execute.

Bibliography

- [1] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Published November 25, 2024. Accessed: 2026-07-05.
- [2] Shi, Jiawen, Yuan, Zenghui, Tie, Guiyao, et al. Prompt injection attack to tool selection in LLM agents. *arXiv preprint arXiv:2504.19793*, 2025.
- [3] Jing, Huihao, Li, Haoran, Hu, Wenbin, et al. MCIP: Protecting MCP safety via model contextual integrity protocol. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. arXiv:2505.14590.
- [4] Zhan, Qiusi, Liang, Zhixiang, Ying, Zifan, and Kang, Daniel. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. arXiv:2403.02691.
- [5] Debenedetti, Edoardo, Zhang, Jie, Balunović, Mislav, et al. Agent-Dojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.
- [6] Bahar, Atmane Ayoub Mansour and Wazan, Ahmad Samer. On the validity of traditional vulnerability scoring systems for adversarial attacks against LLMs. *arXiv preprint arXiv:2412.20087*, 2024.

- [7] Spring, Jonathan M., Hatleback, Eric, Householder, Allen D., Manion, Art, and Shick, Deana. Time to change the CVSS? *IEEE Security & Privacy*, 19(2):74–78, 2021. doi: 10.1109/MSEC.2020.3044475.
- [8] Xing, Wenpeng, Qi, Zhonghao, Qin, Yupeng, Li, Yilin, Chang, Caini, Yu, Jiahui, Lin, Changting, Xie, Zhenzhen, and Han, Meng. MCP-guard: A multi-stage defense-in-depth framework for securing model context protocol in agentic AI. *arXiv preprint arXiv:2508.10991*, 2025.
- [9] Yang, Yixuan, Gao, Cuifeng, Wu, Daoyuan, Chen, Yufan, Li, Yingjiu, and Wang, Shuai. MCPSecBench: A systematic security benchmark and playground for testing model context protocols. *arXiv preprint arXiv:2508.13220*, 2025.
- [10] Model Context Protocol. Security best practices. https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices, 2026. Accessed: 2026-07-05.
- [11] Hou, Xinyi, Zhao, Yanjie, Wang, Shenao, and Wang, Haoyu. Model context protocol (MCP): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*, 2026. doi: 10.1145/3796519. arXiv:2503.23278.
- [12] Rostamzadeh, Mehrdad, Narula, Sidhant, Birhan, Nahom, Ghasemigol, Mohammad, and Takabi, Daniel. MCP-DPT: A defense-placement taxonomy and coverage analysis for model context protocol security. *arXiv preprint arXiv:2604.07551*, 2026.
- [13] Radosevich, Brandon and Halloran, John. MCP safety audit: LLMs with the model context protocol allow major security exploits. *arXiv preprint arXiv:2504.03767*, 2025.
- [14] Hasan, Mohammed Mehedi, Li, Hao, Fallahzadeh, Emad, et al. Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. *arXiv preprint arXiv:2506.13538*, 2025.

- [15] Zhao, Shuli, Hou, Qinsheng, Zhan, Zihan, Wang, Yanhao, Xie, Yuchong, Guo, Yu, Chen, Libo, Li, Shenghong, and Xue, Zhi. Parasites in the toolchain: A large-scale analysis of attacks on the MCP ecosystem. In *IEEE Symposium on Security and Privacy (S&P)*, 2026. arXiv:2509.06572.
- [16] Li, Zhihao, Li, Kun, Ma, Boyang, Xu, Minghui, Zhang, Yue, and Cheng, Xiuzhen. We urgently need privilege management in MCP: A measurement of API usage in MCP ecosystems. In *IEEE International Conference on Mobile Ad-Hoc and Smart Systems (MASS)*. IEEE, 2025. arXiv:2507.06250.
- [17] Bhatt, Manish, Narajala, Vineeth Sai, and Habler, Idan. ETDI: Mitigating tool squatting and rug pull attacks in model context protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control. In *2025 Cyber Awareness and Research Symposium (CARS)*. IEEE, 2025. doi: 10.1109/CARS67163.2025.11337310.
- [18] Bühler, Christoph, Biagiola, Matteo, Di Grazia, Luca, and Salvaneschi, Guido. AgentBound: Securing execution boundaries of AI agents. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2026. arXiv:2510.21236.
- [19] Sharma, Reshabh K, Jiang, Linxi, Lin, Zhiqiang, and Chen, Shuo. PAuth: Precise task-scoped authorization for agents. *arXiv preprint arXiv:2603.17170*, 2026.
- [20] Huang, Yuhang, Ma, Boyang, Yan, Biwei, Dai, Xuelong, Zhang, Yechao, Xu, Minghui, Xu, Kaidi, and Zhang, Yue. Give them an inch and they will take a mile: Understanding and measuring caller identity confusion in MCP-based AI systems. *arXiv preprint arXiv:2603.07473*, 2026.
- [21] Yuan, Tongxin, He, Zhiwei, Dong, Lingzhong, et al. R-Judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. arXiv:2401.10019.

- [22] Andriushchenko, Maksym, Souly, Alexandra, Dziemian, Mateusz, et al. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.09024.
- [23] Ruan, Yangjun, Dong, Honghua, Wang, Andrew, et al. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.15817.
- [24] Cartagena, Arnold and Teixeira, Ariane. Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents. *arXiv preprint arXiv:2602.16943*, 2026.
- [25] Inan, Hakan, Upasani, Kartikeya, Chi, Jianfeng, Rungta, Rashi, Iyer, Krithika, Mao, Yuning, Tontchev, Michael, Hu, Qing, Fuller, Brian, Testuggine, Davide, and Khabsa, Madian. Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- [26] Shi, Tianneng, He, Jingxuan, Wang, Zhun, Li, Hongwei, Wu, Linyu, Guo, Wenbo, and Song, Dawn. Progent: Securing AI agents with privilege control. *arXiv preprint arXiv:2504.11703*, 2025.
- [27] Betser, Roy, Bose, Shamik, Giloni, Amit, Picardi, Chiara, Padakandla, Sindhu, and Vainshtein, Roman. AgentTRIM: Tool risk mitigation for agentic AI. *arXiv preprint arXiv:2601.12449*, 2026.
- [28] Anthropic. Agentic misalignment: How LLMs could be insider threats. <https://www.anthropic.com/research/agentic-misalignment>, 2025. Published June 20, 2025. Accessed: 2026-07-05.
- [29] Jacobs, Jay, Romanosky, Sasha, Edwards, Benjamin, Roytman, Michael, and Adjerid, Idris. Exploit prediction scoring system (EPSS). *Digital Threats: Research and Practice*, 2(3):1–17, 2021. doi: 10.1145/3436242.
- [30] OWASP Foundation. OWASP AI vulnerability scoring system (AIVSS) v0.5. <https://aivss.owasp.org/>, 2025. Accessed: 2026-07-05.

- [31] Mehra, Anket, Aßmuth, Andreas, and Prieß, Malte. Graph of effort: Quantifying risk of AI usage for vulnerability assessment. In *Proceedings of the 16th International Conference on Cloud Computing, GRIDs, and Virtualization (CLOUD COMPUTING)*, 2025. arXiv:2503.16392.
- [32] OWASP Foundation. OWASP top 10 for agentic applications for 2026. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>, 2025. Published December 9, 2025. Accessed: 2026-07-05.
- [33] Malul, Ehud, Meidan, Yair, Mimran, Dudu, Elovici, Yuval, and Shabtai, Asaf. GenKubeSec: LLM-based kubernetes misconfiguration detection, localization, reasoning, and remediation. *arXiv preprint arXiv:2405.19954*, 2024.
- [34] Cohen, Omri Sgan, Malul, Ehud, Meidan, Yair, Mimran, Dudu, Elovici, Yuval, and Shabtai, Asaf. KubeGuard: LLM-assisted kubernetes hardening via configuration files and runtime logs analysis. *arXiv preprint arXiv:2509.04191*, 2025.
- [35] Haddad, Osama Al, Ikram, Muhammad, Ahmed, Ejaz, and Lee, Young. Prompting the priorities: A first look at evaluating LLMs for vulnerability triage and prioritization. *arXiv preprint arXiv:2510.18508*, 2025.
- [36] Liu, Yang, Iter, Dan, Xu, Yichong, Wang, Shuohang, Xu, Ruochen, and Zhu, Chenguang. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2303.16634.
- [37] Zheng, Lianmin, Chiang, Wei-Lin, Sheng, Ying, Zhuang, Siyuan, Wu, Zhanghao, Zhuang, Yonghao, Lin, Zi, Li, Zhuohan, Li, Dacheng, Xing, Eric P., Zhang, Hao, Gonzalez, Joseph E., and Stoica, Ion. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.

- [38] Hossain, Ismail, Puppala, Sai, Alam, Md Jahangir, Ahad, Tanzim, and Talukder, Sajedul. SkillVetBench: LLM-as-judge for multi-dimensional security risk evaluation in open-source LLM agent skills. *arXiv preprint arXiv:2606.15899*, 2026.
- [39] Fleming, Charles, Kundu, Ashish, and Kompella, Ramana. Uncertainty-aware, risk-adaptive access control for agentic systems using an LLM-judged TBAC model. *arXiv preprint arXiv:2510.11414*, 2025.
- [40] Cheimonidis, Pavlos and Rantos, Konstantinos. Dynamic risk assessment in cybersecurity: A systematic literature review. *Future Internet*, 15(10):324, 2023. doi: 10.3390/fi15100324.

תוכן עניינים

vi

רשימת איורים

vii

רשימת טבלאות

1

1 מבוא

1	מוטיבציה	1.1
2	הגדרת הבעיה	1.2
2	גישה ומטרות המחקר	1.3
4	מבנה ההצעה	1.4

5

2 רקע וסקירת ספרות

5	משטח התקיפה של PCM	2.1
6	הגנות והתקפות ייחודיות ל-PCM	2.2
6	בטיחות סוכנים והזרקת פקודות	2.3
7	מעקות בטיחות ושערים בזמן ריצה	2.4
7	ניקוד סיכון: מ-SSVC ועד שופטי MLL	2.5
8	מיקום המחקר המוצע	2.6

9

3 הגישה המוצעת: מסגרת ksiRpcM

9	מודל האיום	3.1
10	ארכיטקטורת המערכת	3.2
12	מודל הניקוד	3.3
13	טקסונומיית הפעולות האטומיות	3.4
14	ניקוד ברמת הפרמטר (בזמן ריצה)	3.5
15	ניקוד שימוש לרעה של יומני קריאות	3.6
17	מימוש ושחזוריות	3.7

18	4 תוצאות ראשוניות	
18	4.1 הגדרות הניסוי	
19	4.2 ניקוד סטטי מול שיפוט אנושי	
20	4.3 יציבות, זמן ריצה וזיהוי מחוץ להתפלגות	
22	5 תוכנית מחקר ותרומות צפויות	
22	5.1 סיכום ההתקדמות	
23	5.2 אתגרים שנוותרו	
23	5.3 עבודה מתוכננת	
24	5.4 לוח זמנים	
24	5.5 תרומות צפויות	
24	ביבליוגרפיה	

הגנה על שרתי PCM מפני סוכנים: מסגרת ניקוד סיכון סטטית ודינמית עבור פרוטוקול ההקשר של המודל

תומר עובדיה

הצעת מחקר לתואר מוסמך למדעי הטבע

אוניברסיטת בן-גוריון בנגב

2026

תקציר

פרוטוקול ההקשר של המודל (PCM) הפך לממשק הסטנדרטי שדרכו סוכני בינה מלאכותית אוטונומיים, המבוססים על מודלי שפה גדולים, מפעילים כלים חיצוניים – מערכות קבצים, מסדי נתונים, מאגרי קוד ופלטפורמות תקשורת. כמעט כל המחקר הקיים בתחום אבטחת PCM עוסק בהגנה על הסוכן מפני שרת זדוני. עבודה זו הופכת את הכיוון ומאמצת את נקודת המבט שבה **השרת הוא הנכס המוגן והסוכן הוא מקור האיום**: לשרת החושף משאבים רגישים אין מנגנון מובנה להחליט, ברגע הגעת הקריאה, האם יש לאשר אותה, לצמצם אותה או לדחות אותה, וסוכן שהוזרק לו פקודות בעקיפין אינו ניתן להבחנה מסוכן לגיטימי.

אנו מתכננים, ממשים ומעריכים את `ksiRpcM`, מסגרת ניקוד סיכון מוכוונת הגנה המשייכת לכל אינטראקציה עם כלי PCM ציון סיכון מדורג ופרשני בארבע רמות. הציון הוא מכפלה של שלושה גורמים פרשניים – רגישות הנכס, רדיוס ההשפעה של הפעולה, והשפעת הכלי (מידת ההפיכות) – כאשר האחרון מעוגן בטקסונומיה של שלוש-עשרה פעולות אטומיות. הציון מחושב סטטית בזמן התכנון ומוסלם בזמן הריצה על-ידי שכבת פרמטרים הקוראת את הארגומנטים בפועל של הקריאה.

הערכנו את המסגרת מול ארבעה סוגי אמת-יסוד: שבעה פריסות PCM מציאותיות, פאנל מומחים ומסגרות בן עשרה מדרגים, מאגר של 969 קריאות סוכן שנלכדו, ו-051 קריאות ממבחני תקיפה חיצוניים. `ksiRpcM` מסכימה עם השיפוט האנושי על השפעת הכלי ב-79 אחוז מהמקרים ומשיגה מקדם קאפא משוקלל של 66.0+ מול הפאנל האנושי

- שני מבין שמונה מסגרות מבוססות ורחוק מעל SSVC - ובניגוד למסגרות מבוססות יכולת בלבד, שומרת על כושר הבחנה בתקיפות חיצוניות חיות. אנו מדווחים גם על חולשות המסגרת, ובעיקר על כך ש-58 אחוז מהתאים שקיבלו ציון רגשים לשינוי של יחידה אחת בקלט, וגוזרים מהמדידות הללו סדר-יום קונקרטי לשער סיכון צד-שרת בר-פריסה.



אוניברסיטת בן-גוריון בנגב
הפקולטה למדעי הטבע
המחלקה למדעי המחשב

הגנה על שרתי PCM מפני סוכנים: מסגרת ניקוד סיכון סטטית ודינמית עבור פרוטוקול ההקשר של המודל

הצעת מחקר לקראת קבלת התואר מוסמך למדעי הטבע (M.Sc)

תומר עובדיה

בהנחיית פרופ' אסף שבתאי

יולי 2026